

# ReviewPulse — Product Roadmap

**The only mobile-native review manager for solo professionals.** Send SMS review requests after every job, monitor Google/Yelp in real time, respond with AI drafts. \$14.99/month, no contract, cancel anytime.

## 1. Executive Summary

### What

ReviewPulse is a mobile-first review management platform for solo business owners — tradespeople, beauty professionals, wellness practitioners, and other service providers who depend on Google reviews for new customers. Users send SMS review requests directly from their phone after completing a job, monitor incoming reviews in real time, and reply with one-tap AI-drafted responses.

### Why

- 28.5M solo businesses in the US rely on Google reviews for discovery
- Google's own "Ask for Reviews" tool is desktop-only
- Competitors charge \$75-600/month with predatory annual contracts
- No mobile-native review manager exists in the market
- SMS has 98% open rate vs 20% email — the right channel for review collection

### For Whom

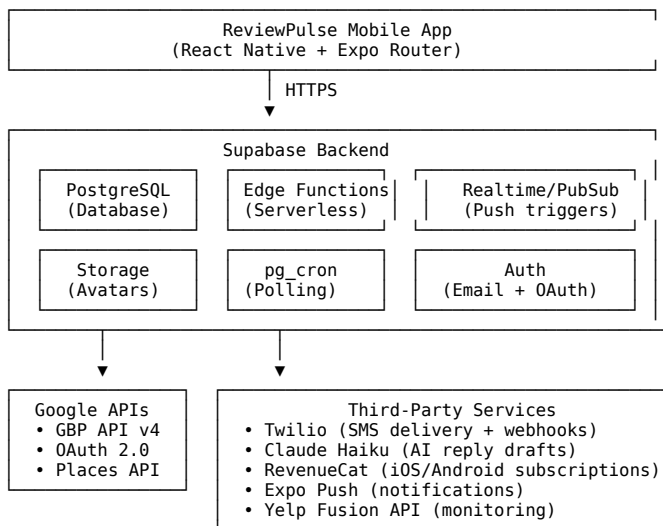
**Primary:** Solo home service contractors (plumbers, electricians, HVAC, roofers, landscapers, handymen) — 3.5M US operators **Secondary:** Beauty & personal care (barbers, hair stylists, nail techs, estheticians) — 1.5M US operators **Tertiary:** Health/wellness, automotive, food & beverage, professional services — 4M+ US operators

### Monetization

- **Free Tier:** Connect 1 Google Business Profile, view all reviews, 5 SMS requests/month, basic analytics
- **Pro Tier:** \$14.99/month — unlimited SMS requests, AI-drafted replies, Yelp/Trustpilot monitoring, full analytics, push notifications, bulk CSV import, no annual contract
- **Revenue target:** 1,000 Pro subscribers = \$14,990/month MRR

## 2. Technical Architecture

### System Overview



### Data Flow: Review Monitoring

1. pg\_cron runs every 15 minutes on Supabase
2. Edge Function: poll-reviews reads all active business\_profiles
3. For each profile: call GBP API accounts.locations.reviews.list
4. Upsert new reviews to reviews table (dedup by platform\_review\_id)
5. New reviews trigger Supabase NOTIFY → Realtime → Edge Function
6. send-push-notification Edge Function sends Expo push notification
7. Mobile app receives notification, updates local review feed

### Data Flow: SMS Review Request

1. User taps "New Request" in app
2. Enters customer name + phone number
3. Selects template (pre-built or custom)

- 3/11/26, 9:13 PM
- 4. App calls send-sms Edge Function via Supabase RPC
  - 5. Edge Function validates: user Pro status, SMS quota, opt-out list
  - 6. Builds SMS content with review link:  
https://search.google.com/local/writereview?placeid=ChIJ...
  - 7. Calls Twilio API (API key stored in Edge Function env, never client-side)
  - 8. Stores review\_request record with status=sent, twilio\_sid
  - 9. Twilio delivery receipt updates status via webhook
  - 10. User sees request in "Sent" list with delivery status

Data Flow: AI Reply Draft

- 1. User taps "Draft Reply" on a review
- 2. App calls generate-reply Edge Function via Supabase RPC
- 3. Edge Function reads: review content, rating, business category, tone preference
- 4. Calls Claude Haiku API with structured prompt
- 5. Returns draft reply text (100-150 words)
- 6. User sees draft pre-populated in reply text field
- 7. User edits (optional) and taps "Post Reply"
- 8. App calls reply-to-review Edge Function
- 9. Edge Function calls GBP API accounts.locations.reviews.reply
- 10. Updates reviews.reply\_content and reviews.replied\_at

Performance Requirements

| Operation                    | Target Latency  | Approach                |
|------------------------------|-----------------|-------------------------|
| Review feed load             | <800ms          | Pagination, local cache |
| New review push notification | <15 min         | pg_cron poll interval   |
| SMS send                     | <3s UI response | Edge Function + Twilio  |
| AI draft generation          | <4s             | Claude Haiku streaming  |
| Review reply post            | <3s             | Direct GBP API call     |
| Analytics chart load         | <1s             | Pre-computed aggregates |

3. Screen-by-Screen Specification

FLOW 1: Authentication

Screen: Welcome (auth/welcome)

**Purpose:** First impression, value proposition, convert to signup **Elements:** - App logo (ReviewPulse star icon with pulse animation) - Hero headline: "More reviews. Less effort." - Subtitle: "Send SMS requests, monitor every platform, reply with AI. Built for solo businesses." - Social proof: "Join 1,000+ business owners getting more reviews" - Primary CTA: "Get Started Free" → onboarding flow - Secondary: "Sign In" → sign-in screen - No scroll required — everything above the fold

**States:** - Default: animated star logo pulses gently - Loading: n/a (static screen)

Screen: Sign In (auth/sign-in)

**Purpose:** Returning user login **Elements:** - Logo (small, top center) - Email input with keyboard dismiss handling - Password input with show/hide toggle - "Forgot Password?" link - Sign In button (primary, full-width) - Divider: "or" - Apple Sign In button (required by App Store for apps with login) - "Don't have an account? Get Started" link - Bottom safe area padding

**States:** - Default: empty form - Filling: inline validation on blur (email format, password min length) - Submitting: button shows loading spinner, fields disabled - Error: red error banner below fields ("Invalid email or password") - Success: navigate to (tabs) or onboarding

Screen: Sign Up (auth/sign-up)

**Purpose:** New user registration **Elements:** - Business name input (required) - Email input - Password input (min 8 chars) - Terms of Service + Privacy Policy checkbox - Create Account button - Apple Sign In alternative - Already have account link

**States:** Same pattern as sign-in, plus password strength indicator

Screen: Forgot Password (auth/forgot-password)

**Purpose:** Reset password via email **Elements:** - Email input - "Send Reset Link" button - Success state: "Check your email" confirmation - Back to Sign In link

FLOW 2: Onboarding (first-time setup)

Screen: Business Type (onboarding/business-type)

**Purpose:** Segment user for personalized templates and UI **Elements:** - Progress indicator (step 1 of 3) - Headline: "What type of business do you run?" - Grid of 8 category cards with icons: - Trades & Home Services (wrench icon) - Beauty & Hair (scissors icon) - Health & Wellness (heart icon) - Food & Beverage (coffee icon) - Auto Services (car icon) - Professional Services (briefcase icon) - Real Estate (home icon) - Other (grid icon) - Continue button (active after selection) - Skip text link (small, below button)

**State management:** Saves to profiles.business\_category

Screen: Connect Google (onboarding/connect-google)

**Purpose:** OAuth flow to link Google Business Profile **Elements:** - Progress indicator (step 2 of 3) - Headline: “Connect your Google Business Profile” - Explanation: “We’ll monitor your reviews and help you reply. We only read reviews and post replies — nothing else.” - Permission bullet points: - ✓ Read your Google reviews - ✓ Post replies on your behalf - ✓ View your business name and rating - ✗ Access personal Google account - ✗ Post to Google Maps or Search - Connect with Google button (white with Google logo) - “Skip for now” text link - Privacy reassurance footer

**OAuth flow:** expo-auth-session → Google OAuth 2.0 → GBP API scope: <https://www.googleapis.com/auth/business.manage>

**States:** - Default: connect button active - Connecting: WebBrowser opens Google OAuth page - Success: brief success animation → auto-advance if 1 location found - Multiple locations: show location picker (FlatList of business locations) - Error: “Couldn’t connect” error with retry

### Screen: First Request Prompt (onboarding/first-request)

**Purpose:** Show immediate value — send first review request **Elements:** - Progress indicator (step 3 of 3) - Headline: “Send your first review request” - Subtext: “Enter a recent customer’s info and we’ll send them a review request right now.” - Customer name input - Phone number input (with country code selector, default +1 US) - Preview of SMS message (non-editable, shows default template) - “Send Now” button - “I’ll do this later” skip link

**States:** - Default: empty inputs - Filled: SMS preview updates with customer name - Sending: button loading state - Sent: success animation + “Done! Let’s go to your dashboard” CTA

## FLOW 3: Main App (tab navigation)

### Screen: Dashboard / Review Feed (tabs/index)

**Purpose:** Primary daily-use screen — see all new reviews, act on them **Header:** “ReviewPulse” logo left, notification bell right (badge for unread), filter button **Content:**

**Rating Summary Card** (top, pinned above scroll): - Large rating number (e.g., “4.8”) with star row - “↑ 0.2 this month” trend indicator - Total review count - “12 unanswered” badge (pro feature) - Platform tabs: Google | Yelp | All

**Review List** (FlatList): - Each review card shows: - Author avatar (initial letter circle) - Author name + date (relative: “2 days ago”) - Star rating (1-5 filled stars) - Review snippet (2 lines, tap to expand) - Platform badge (Google/Yelp pill) - Reply status badge: “Replied” (green) / “Not Replied” (amber) / “Flagged” (red) - Action row: “Draft Reply” | “Flag” | “Share”

**Empty State:** - Illustration of a phone receiving a star - “No reviews yet” headline - “Send your first review request to get started” subtext - “Send Request” CTA button

**Loading State:** - 3 skeleton cards pulsing

**States:** - Default: populated review list, newest first - Refreshing: pull-to-refresh spinner - Filtering: by platform or rating - New review badge: “3 new” indicator on tab

### Screen: Review Detail (review/[id])

**Purpose:** Read full review, draft/post reply **Navigation:** Push from review card tap **Header:** Back arrow, review date, “Share” icon

**Content:** - Author name + avatar - Star rating (large, visual) - Full review text (no truncation) - Date posted - Platform badge

**Reply Section:** - “Your Reply” heading - If already replied: green “Replied” badge + reply text (grayed, non-editable) - If not replied: - “Draft with AI” button (primary, full-width) — PRO - Text area (placeholder: “Write a professional response...”) - Character counter (max 4096 chars for Google) - “Post Reply” button

**Flag Section:** - “Suspicious review?” toggle - If flagged: “Report to Google” button - Flag reasons: Fake | Spam | Irrelevant | Former Employee | Competitor

**AI Draft Loading State:** - Shimmer animation in text area while draft generating - “AI is crafting your response...” text

### Screen: Requests Tab (tabs/requests)

**Purpose:** Manage review request sending and tracking **Header:** “Review Requests”, “New Request” button (+ icon)

**Stats Bar** (if Pro): - “Sent this month: 12” | “Delivery rate: 94%” | “Est. reviews: 3” - If Free: “5/5 requests used this month” + “Upgrade for unlimited” banner

**Request List** (FlatList with section headers by status): - Sections: “Awaiting” | “Sent” | “Delivered” | “Clicked” | “Completed” - Each row: Customer name | Phone | Date sent | Status badge | Resend button (if failed)

**Empty State:** - Illustration: phone sending a message with a star - “No requests yet” - “Tap + to send your first review request” CTA

**FAB** (Floating Action Button): Large + button, green, springs in on mount

### Screen: New Request (request/new)

**Purpose:** Create and send a review request **Navigation:** Modal sheet (rises from bottom) **Header:** “New Request” title, × close button

**Form:** - Customer Name (required, autofocus) - Phone Number (required, formatted with dashes as user types) - OR Choose from contacts button (opens customer list picker) - Platform selector: Google | Yelp | Both - Template selector (expandable): Default | HIPAA-Aware | Short & Sweet | Long & Grateful | [Custom templates] - Message preview (live-updating as template + fields change) - Character count

**Send Button:** - Free tier: “Send (X of 5 remaining)” - Pro tier: “Send Request” - Pressed: scale 0.97 + haptic medium

**Validation:** - Name: required - Phone: valid US format (10 digits) or international - Inline error messages

### Screen: Analytics Tab (tabs/analytics)

**Purpose:** Track review growth, response rate, velocity **Header:** “Analytics” | Time range picker (30d | 90d | 1y)

**Pro Gate** (if Free): - Blurred chart with “Pro” badge overlay - “Unlock full analytics” CTA button

**Charts** (Pro only): 1. Rating Trend line chart — average rating over time 2. Review Velocity bar chart — new reviews per week 3. Response Rate ring chart — % of reviews replied 4. Platform breakdown pie — Google vs Yelp vs others

**Stats Grid** (2x2): - Total Reviews | New This Month - Average Rating | Response Rate

**Insights Panel** (AI-generated, 1-2 sentences): - “Your rating improved 0.3 stars this month — your best month yet” - “You have 8 unanswered negative reviews from the last 90 days”

**Export Button** (Pro): “Export CSV” — sends data file to email

### Screen: Settings Tab (tabs/settings)

**Purpose:** Account management, connected platforms, notifications, subscription **Header:** “Settings”

**Sections:** 1. **Account** - Profile photo + name (tappable → edit) - Business name - Business category - Sign out

2. **Connected Platforms**

- Google Business Profile: connected (green dot) / disconnected (gray) + “Reconnect” or “Connect”
- Yelp: connected / “Connect” (Pro only)
- Trustpilot: “Coming soon” (grayed)

3. **Notifications**

- New review alert: toggle
- Negative review (1-2 stars) urgent alert: toggle
- Weekly summary: toggle
- Request delivered confirmation: toggle

4. **Subscription**

- Free tier: “Upgrade to Pro” card (prominent, shows feature list)
- Pro tier: “ReviewPulse Pro — Active” + “Manage Subscription” (RevenueCat portal)
- “Restore Purchases” link

5. **Templates**

- “Manage Templates” → templates screen

6. **Customers**

- “Manage Customer List” → customers screen
- “Import Contacts (CSV)” → CSV picker

7. **About**

- App version
- Privacy Policy
- Terms of Service
- Contact Support

---

## FLOW 4: Paywall

### Screen: Paywall (paywall)

**Purpose:** Convert free users to Pro **Trigger:** Attempting any Pro feature (AI draft, 6th SMS, Yelp monitoring, etc.)

**Layout** (full-screen modal): - Close button (× top right, subtle) - “Upgrade to ReviewPulse Pro” headline - Subheading: “Get 10x more reviews without the agency price tag”

**Feature List** (checkmark rows): - ✓ Unlimited SMS review requests - ✓ AI-drafted reply suggestions - ✓ Yelp & Trustpilot monitoring - ✓ Full analytics + insights - ✓ Push notifications for every review - ✓ Custom SMS templates - ✓ Bulk import customer contacts - ✓ Suspicious review flagging

**Pricing Toggle:** Monthly | Annual (save 30%) - Monthly: \$14.99/month - Annual: \$9.99/month (\$119.99/year)

**CTA Button:** “Start Free Trial — 7 Days” (primary, large) - Below: “Then \$14.99/month. Cancel anytime.”

**Social Proof:** - 3 testimonial cards (carousel): - “I went from 12 to 47 Google reviews in 6 weeks.” — Mike T., Plumber - “Sent 25 requests, got 11 reviews. Worth every penny.” — Sarah K., Esthetician - “Best \$15 I spend on my business each month.” — Carlos R., Auto Detailer

**Restore Purchases** link (small, bottom)

---

## FLOW 5: Customer Management

### Screen: Customer List (customers/index)

**Purpose:** Manage contact list for review requests **Header:** “Customers”, Search bar, Import button

**List:** Customer rows showing name, phone, last request date, review count **Swipe actions:** Archive (left) | Edit (right) **Empty state:** “Add your first customer” CTA

### Screen: Customer Detail (customers/[id])

**Purpose:** View history, send request **Elements:** Contact info, request history timeline, notes field, “Send Request” button

---

## FLOW 6: Templates

**Screen: Template List (templates/index)**

**Purpose:** Manage SMS templates **Header:** “Templates”, + New Template button

**Sections:** System Templates | My Templates

**System template categories:** - Default (trades) - HIPAA-Aware (healthcare) - Beauty & Style - Friendly & Short - Post-Job Thank You - Formal Professional

**Custom templates:** user-created, editable, deletable

**Screen: Template Editor (templates/[id] or templates/new)**

**Purpose:** Create or edit SMS template **Elements:** - Template name input - Message body textarea (with merge tag buttons: {customer\_name}, {business\_name}, {review\_link}) - Character counter (160-char SMS = 1 message; warn at 160, error at 320) - Platform selector - Industry tag - Preview of final message - Save button

## 4. Feature Breakdown

### Feature 1: Google Business Profile OAuth Connection

**What it does:** Authenticates user’s Google account with GBP API scope, fetches and stores their business location(s), begins review monitoring.

**Acceptance criteria:** - User taps “Connect Google” and is redirected to Google OAuth consent screen - App requests only `business.manage` scope (read reviews + post replies) - On success, app fetches all Google Business locations for the account - If 1 location: auto-selects, saves to `business_profiles` - If multiple: shows picker UI, user selects primary location - Stores encrypted `access_token` and `refresh_token` in Supabase - Shows connected state with business name and current rating

**Edge cases:** - User denies permission: show explanation modal with retry - Account has no GBP listing: show “Set up Google Business Profile” guide link - Token expires: background refresh via Edge Function, prompt reconnect if refresh fails - User connects wrong account: allow disconnect + reconnect

**Technical approach:** - expo-auth-session with `useAuthRequest` hook - Authorization URL: `accounts.google.com/o/oauth2/v2/auth` - Scopes: `https://www.googleapis.com/auth/business.manage` - Token exchange in Supabase Edge Function (never client-side) - Tokens stored in `business_profiles.google_access_token` (AES-256 encrypted) - `google-auth-library` Node package in Edge Function for token refresh

**Complexity:** L

### Feature 2: Review Monitoring (Google)

**What it does:** Polls GBP API every 15 minutes, syncs new reviews to database, triggers push notifications.

**Acceptance criteria:** - New reviews appear in feed within 15 minutes of being posted on Google - Reviews show author name, rating, content, date, reply status - Pull-to-refresh triggers immediate manual sync - “Last synced” timestamp shown in dashboard header - Handles pagination (GBP API returns max 50 reviews per call)

**Edge cases:** - Review deleted from Google: mark as deleted in DB, hide from feed (keep for analytics) - API rate limit hit: exponential backoff, retry in next cycle - Business profile reconnect needed: notification with “Reconnect” deep link - No reviews yet: empty state (not error state)

**Technical approach:** - `pg_cron` job: every 15 minutes, calls `poll-reviews` Edge Function - Edge Function calls `mybusiness.googleapis.com/v4/{accountId}/locations/{locationId}/reviews` - Upsert into `reviews` table using `platform_review_id` as conflict key - New reviews inserts trigger Postgres NOTIFY → Supabase Realtime → push notification Edge Function - Client subscribes to Supabase Realtime channel for live review feed updates

**Complexity:** L

### Feature 3: SMS Review Requests

**What it does:** Sends a personalized SMS to a customer with a direct link to leave a Google review.

**Acceptance criteria:** - User enters customer name + phone number - App generates review link: `https://search.google.com/local/writereview?placeid={PLACE_ID}` - SMS delivered within 5 seconds of user tapping Send - Customer receives: “Hi [Name], thanks for choosing [Business]! We’d appreciate a quick review: [link] – Reply STOP to opt out.” - Delivery receipt tracked via Twilio webhook - Free tier: 5 SMS/month enforced server-side - Opt-out respected: STOP replies update `customers.opt_out`

**Edge cases:** - Invalid phone number: Twilio returns 400, show “Invalid phone number” error - Opt-out customer: block send, show “Customer opted out” badge - Twilio delivery failure: `status=failed`, show retry button - Free tier exhausted: block send, show paywall

**Technical approach:** - `send-sms` Supabase Edge Function called via RPC - Edge Function: validate quota → check opt-out → generate link → call Twilio Messages API - Twilio webhook endpoint: `supabase/functions/twilio-webhook` — handles delivery receipts + STOP replies - Twilio sender: verified US long-code number (provisioned via Twilio console) - SMS content max 160 chars (1 segment) — template enforces this

**Compliance notes:** - TCPA compliance: user must confirm they have customer consent before sending - Include opt-out instructions in every message (Reply STOP) - Store consent timestamp in `review_requests` table

**Complexity:** L

### Feature 4: AI Reply Drafts

**What it does:** Generates a professional, contextually appropriate reply to a customer review using Claude Haiku.

**Acceptance criteria:** - “Draft Reply” button appears on every unreplied review (Pro only) - Draft generated in < 4 seconds - Draft is 80-120 words, professional, addresses the specific content - 1-star draft: empathetic acknowledgment + invitation to discuss - 5-star draft: warm thank-you, mention specific details from review - User can edit draft before posting - Draft saved to reviews.ai\_draft column (persists if user navigates away)

**Edge cases:** - Empty review (rating only, no text): generate generic response based on rating - Review in non-English language: detect language, respond in same language - Sensitive content in review: safety filters prevent problematic drafts - API timeout: fallback message + retry button - Very long review: truncate input to 500 chars (still captures key sentiment)

**Technical approach:** - generate-reply Supabase Edge Function called via RPC - Prompt structure: `` You are a professional business owner responding to a customer review. Business: {business\_name} ({business\_category}) Review rating: {rating}/5 stars Review content: “{content}”

Write a professional, genuine response in 80-120 words. - Personalize to the specific content of the review - For positive reviews: warm thank-you, specific acknowledgment - For negative reviews: empathy, apology if appropriate, offer to resolve - Never be defensive or dismissive - End with your first name or business name - No marketing language or hashtags `` - Model: claude-haiku-4-5-20251001` - Temperature: 0.7 (slight creativity for naturalness) - Max tokens: 200

**Complexity:** M

---

## Feature 5: Review Reply Posting

**What it does:** Posts user’s reply directly to Google reviews via GBP API.

**Acceptance criteria:** - “Post Reply” button submits reply to Google within 5 seconds - Success: review card shows “Replied” badge, reply text visible in feed - Failure: error message with retry option (reply text preserved) - Cannot reply twice (button hidden after reply posted) - Reply character limit: 4096 chars (Google limit)

**Technical approach:** - post-reply Supabase Edge Function - Calls mybusiness.googleapis.com/v4/{accountId}/locations/{locationId}/reviews/{reviewId}/reply - Method: PUT with {comment: "reply text"} - Updates reviews.reply\_content, reviews.replied\_at, reviews.is\_replied = true - Handles 401 (token expired): triggers token refresh, retries once

**Complexity:** S

---

## Feature 6: Push Notifications

**What it does:** Alerts user on their device when new reviews arrive.

**Acceptance criteria:** - Notification arrives within 15 minutes of review being posted - Notification: “[Author Name] left you a [N]-star review: [excerpt]...” - Tapping notification deep-links to review detail screen - 1-2 star reviews get urgent notification (different sound/badge) - User can configure notification types in settings

**Technical approach:** - expo-notifications for permission request + token registration - Push token saved to push\_tokens table on first launch - Supabase Realtime subscription in Edge Function monitors reviews inserts - send-push-notification Edge Function calls Expo Push API - Notification payload includes reviewId for deep link

**Complexity:** M

---

## Feature 7: Analytics Dashboard

**What it does:** Shows review growth trends, response rate, and platform breakdown over time.

**Acceptance criteria** (Pro only): - Rating trend chart: line chart of average rating per week - Review velocity: bar chart of new reviews per week - Response rate: donut chart of replied vs unreplied - Platform breakdown: pie chart - All charts support 30-day, 90-day, 1-year time ranges - Insights panel shows 1-2 AI-generated actionable observations

**Technical approach:** - Pre-computed aggregates via Supabase materialized view (refreshed hourly) - react-native-gifted-charts for all chart types - Insights generated by small Claude Haiku prompt: Given these stats: rating\_delta={x}, response\_rate={y}%, recent\_negatives={n}, write ONE insight sentence (max 120 chars) a business owner would find useful.

**Complexity:** M

---

## Feature 8: Subscription / RevenueCat Integration

**What it does:** Manages Pro subscription via RevenueCat, gates premium features.

**Acceptance criteria:** - In-app purchase available: \$14.99/month or \$119.99/year - 7-day free trial - Paywall shown when attempting Pro features (not on app open) - Restore Purchases works for users who reinstalled - Subscription status synced to profiles.is\_pro via RevenueCat webhook - Grace period: 3-day grace if payment fails before downgrading

**Technical approach:** - react-native-purchases (RevenueCat SDK) - Entitlement: pro — maps to both monthly and annual products - subscription Zustand store: reads from RevenueCat on app start - Pro gate hook: useIsPro() — returns true/false - Server-side validation: RevenueCat webhook updates profiles.is\_pro - RevenueCat webhook endpoint: supabase/functions/revenuecat-webhook

**Products to configure in App Store Connect + Play Store:** - reviewpulse\_pro\_monthly: \$14.99/month, 7-day trial - reviewpulse\_pro\_annual: \$119.99/year, 7-day trial

**Complexity:** M

---

## Feature 9: Yelp Monitoring (Pro)

**What it does:** Monitors Yelp business profile for new reviews.

**Acceptance criteria** (Pro only): - User connects Yelp via “Find my business” (uses Yelp Fusion API search by name/address) - New Yelp reviews sync with same 15-minute polling - Yelp reviews appear in same feed with Yelp badge - Cannot reply to Yelp reviews in-app (Yelp API read-only; show “Reply on Yelp” link)

**Technical approach:** - Yelp Fusion API: /businesses/search to find business by name - Store yelp\_alias in business\_profiles - Poll /businesses/{alias}/reviews every 15 min - Yelp API key stored in Edge Function environment (never client-side)

**Complexity:** M

## Feature 10: Customer List Management

**What it does:** Maintains a contact database for reusing customer info for review requests.

**Acceptance criteria:** - After sending first request, option to save customer to list - Browse and search saved customers - Tap a customer to send new request with their info pre-filled - CSV import: upload a spreadsheet of customers (name + phone) - Opt-out customers clearly marked, cannot be sent requests

**Technical approach:** - expo-document-picker for CSV file selection - CSV parsing via papaparse (runs client-side) - Batch upsert to customers table via Supabase SDK - Opt-out list joined on every request send check

**Complexity:** M

## 5. Database Schema

```
-- =====
-- ReviewPulse Supabase Database Schema
-- =====

-- Enable necessary extensions
CREATE EXTENSION IF NOT EXISTS "pgcrypto";

-- PROFILES (extends auth.users)
-- =====
CREATE TABLE public.profiles (
  id                UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  business_name     TEXT,
  owner_name        TEXT,
  business_category TEXT CHECK (business_category IN (
    'trades', 'beauty', 'wellness', 'food', 'automotive',
    'professional', 'real_estate', 'other'
  )),
  phone             TEXT,
  avatar_url        TEXT,
  is_pro            BOOLEAN NOT NULL DEFAULT FALSE,
  revenuecat_id     TEXT,
  sms_count_this_month INTEGER NOT NULL DEFAULT 0,
  sms_reset_at      TIMESTAMPTZ NOT NULL DEFAULT date_trunc('month', NOW()),
  onboarding_complete BOOLEAN NOT NULL DEFAULT FALSE,
  created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Auto-create profile on signup
CREATE OR REPLACE FUNCTION public.handle_new_user()
RETURNS TRIGGER AS $$
BEGIN
  INSERT INTO public.profiles (id, owner_name)
  VALUES (NEW.id, NEW.raw_user_meta_data->>'full_name');
  RETURN NEW;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

CREATE TRIGGER on_auth_user_created
  AFTER INSERT ON auth.users
  FOR EACH ROW EXECUTE FUNCTION public.handle_new_user();

-- RLS
ALTER TABLE public.profiles ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Users can view own profile"
  ON public.profiles FOR SELECT USING (auth.uid() = id);
CREATE POLICY "Users can update own profile"
  ON public.profiles FOR UPDATE USING (auth.uid() = id);

-- BUSINESS PROFILES (connected platforms)
-- =====
CREATE TABLE public.business_profiles (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id           UUID NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,
  platform          TEXT NOT NULL CHECK (platform IN ('google', 'yelp', 'trustpilot')),
  platform_id       TEXT NOT NULL, -- google place_id, yelp alias
  business_name     TEXT,
  address           TEXT,
  phone             TEXT,
  -- Encrypted tokens (AES-256 via pgcrypto)
  google_access_token TEXT,
  google_refresh_token TEXT,
  google_token_expiry TIMESTAMPTZ,
  google_account_id  TEXT,
  google_location_id TEXT,
  -- Yelp
```

3/11/26, 9:13 PM

```
yelp_api_key      TEXT,
-- Status
current_rating    DECIMAL(2,1),
review_count      INTEGER DEFAULT 0,
last_polled_at    TIMESTAMPTZ,
poll_error_count  INTEGER DEFAULT 0,
is_active         BOOLEAN NOT NULL DEFAULT TRUE,
connected_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
UNIQUE(user_id, platform, platform_id)
);

ALTER TABLE public.business_profiles ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Users can manage own business profiles"
  ON public.business_profiles FOR ALL USING (auth.uid() = user_id);

-----
-- REVIEWS
-----
CREATE TABLE public.reviews (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  business_profile_id  UUID NOT NULL REFERENCES public.business_profiles(id) ON DELETE CASCADE,
  user_id           UUID NOT NULL REFERENCES public.profiles(id),
  platform_review_id TEXT NOT NULL,
  platform          TEXT NOT NULL CHECK (platform IN ('google', 'yelp', 'trustpilot')),
  author_name       TEXT,
  author_photo_url  TEXT,
  rating            INTEGER CHECK (rating BETWEEN 1 AND 5),
  content           TEXT,
  published_at      TIMESTAMPTZ,
  -- Reply
  reply_content     TEXT,
  replied_at        TIMESTAMPTZ,
  is_replied        BOOLEAN NOT NULL DEFAULT FALSE,
  ai_draft          TEXT,
  -- Status
  is_flagged        BOOLEAN NOT NULL DEFAULT FALSE,
  flag_reason       TEXT,
  is_deleted        BOOLEAN NOT NULL DEFAULT FALSE,
  sentiment         TEXT CHECK (sentiment IN ('positive', 'neutral', 'negative')),
  -- Metadata
  synced_at         TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE(business_profile_id, platform_review_id)
);

-- Index for feed query
CREATE INDEX idx_reviews_user_id_published ON public.reviews(user_id, published_at DESC);
CREATE INDEX idx_reviews_unreplied ON public.reviews(user_id, is_replied, is_deleted) WHERE is_replied = FALSE AND is_deleted = FALSE;

ALTER TABLE public.reviews ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Users can manage own reviews"
  ON public.reviews FOR ALL USING (auth.uid() = user_id);

-----
-- CUSTOMERS (contact list)
-----
CREATE TABLE public.customers (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id           UUID NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,
  name              TEXT NOT NULL,
  phone             TEXT,
  email             TEXT,
  opt_out           BOOLEAN NOT NULL DEFAULT FALSE,
  opt_out_at        TIMESTAMPTZ,
  request_count     INTEGER NOT NULL DEFAULT 0,
  last_request_at   TIMESTAMPTZ,
  tags              TEXT[],
  notes             TEXT,
  created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_customers_user_id ON public.customers(user_id);
CREATE INDEX idx_customers_phone ON public.customers(user_id, phone);

ALTER TABLE public.customers ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Users can manage own customers"
  ON public.customers FOR ALL USING (auth.uid() = user_id);

-----
-- TEMPLATES
-----
CREATE TABLE public.templates (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id           UUID REFERENCES public.profiles(id) ON DELETE CASCADE, -- NULL = system template
  name              TEXT NOT NULL,
  content           TEXT NOT NULL, -- {customer_name}, {business_name}, {review_link}
  platform          TEXT NOT NULL DEFAULT 'google',
  industry          TEXT,
  is_system         BOOLEAN NOT NULL DEFAULT FALSE,
  char_count        INTEGER GENERATED ALWAYS AS (length(content)) STORED,
  created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

ALTER TABLE public.templates ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Users can read system templates and own templates"
  ON public.templates FOR SELECT USING (user_id IS NULL OR auth.uid() = user_id);
```



```

CREATE POLICY "Users can manage own templates"
  ON public.templates FOR INSERT WITH CHECK (auth.uid() = user_id);
CREATE POLICY "Users can update own templates"
  ON public.templates FOR UPDATE USING (auth.uid() = user_id AND is_system = FALSE);
CREATE POLICY "Users can delete own templates"
  ON public.templates FOR DELETE USING (auth.uid() = user_id AND is_system = FALSE);

-- Insert system templates
INSERT INTO public.templates (id, user_id, name, content, platform, industry, is_system) VALUES
  (gen_random_uuid(), NULL, 'Default (Trades)', 'Hi {customer_name}, thanks for choosing {business_name}! We'd love to hear your feedback -
  (gen_random_uuid(), NULL, 'Short & Sweet', 'Hey {customer_name}! Mind leaving us a quick review? It takes 30 seconds: {review_link} Thanks!
  (gen_random_uuid(), NULL, 'HIPAA-Aware (Healthcare)', 'Hi {customer_name}, thank you for visiting {business_name}. If you had a positive ex
  (gen_random_uuid(), NULL, 'Beauty & Style', 'Hi {customer_name}! Hope you're loving your results ☺ If you have a moment, a quick review w
  (gen_random_uuid(), NULL, 'Professional (Formal)', 'Dear {customer_name}, thank you for choosing {business_name}. Your feedback is importan

-- REVIEW REQUESTS
CREATE TABLE public.review_requests (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id           UUID NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,
  customer_id       UUID REFERENCES public.customers(id) ON DELETE SET NULL,
  customer_name     TEXT NOT NULL,
  customer_phone    TEXT,
  customer_email    TEXT,
  platform          TEXT NOT NULL DEFAULT 'google',
  review_link       TEXT NOT NULL,
  template_id       UUID REFERENCES public.templates(id) ON DELETE SET NULL,
  message_content   TEXT NOT NULL,
  channel           TEXT NOT NULL DEFAULT 'sms',
  status            TEXT NOT NULL DEFAULT 'pending' CHECK (status IN (
    'pending', 'sent', 'delivered', 'clicked', 'reviewed', 'failed', 'opted_out'
  )),
  twilio_message_sid TEXT,
  sent_at           TIMESTAMPTZ,
  delivered_at      TIMESTAMPTZ,
  clicked_at        TIMESTAMPTZ,
  error_message     TEXT,
  created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_review_requests_user_id ON public.review_requests(user_id, created_at DESC);
CREATE INDEX idx_review_requests_twilio ON public.review_requests(twilio_message_sid) WHERE twilio_message_sid IS NOT NULL;

ALTER TABLE public.review_requests ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Users can manage own requests"
  ON public.review_requests FOR ALL USING (auth.uid() = user_id);

-- NOTIFICATIONS (in-app)
CREATE TABLE public.notifications (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id           UUID NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,
  type              TEXT NOT NULL CHECK (type IN (
    'new_review', 'negative_review', 'request_delivered',
    'request_clicked', 'weekly_summary', 'subscription_expiring'
  )),
  title             TEXT NOT NULL,
  body              TEXT,
  data              JSONB DEFAULT '{}',
  review_id         UUID REFERENCES public.reviews(id) ON DELETE SET NULL,
  read_at           TIMESTAMPTZ,
  created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_notifications_user_unread ON public.notifications(user_id, read_at) WHERE read_at IS NULL;

ALTER TABLE public.notifications ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Users can manage own notifications"
  ON public.notifications FOR ALL USING (auth.uid() = user_id);

-- PUSH TOKENS
CREATE TABLE public.push_tokens (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id           UUID NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,
  token             TEXT NOT NULL UNIQUE,
  platform          TEXT NOT NULL DEFAULT 'ios',
  device_id        TEXT,
  is_active         BOOLEAN NOT NULL DEFAULT TRUE,
  created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  last_used_at      TIMESTAMPTZ
);

ALTER TABLE public.push_tokens ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Users can manage own push tokens"
  ON public.push_tokens FOR ALL USING (auth.uid() = user_id);

-- ANALYTICS MATERIALIZED VIEW (refresh hourly)
CREATE MATERIALIZED VIEW public.review_analytics AS
SELECT
  r.user_id,
  date_trunc('week', r.published_at) AS week,
  AVG(r.rating)::DECIMAL(3,2) AS avg_rating,

```

```

COUNT(*) AS review_count,
COUNT(*) FILTER (WHERE r.is_replied) AS replied_count,
COUNT(*) FILTER (WHERE r.rating <= 2) AS negative_count,
COUNT(*) FILTER (WHERE r.platform = 'google') AS google_count,
COUNT(*) FILTER (WHERE r.platform = 'yelp') AS yelp_count
FROM public.reviews r
WHERE r.is_deleted = FALSE
AND r.published_at IS NOT NULL
GROUP BY r.user_id, date_trunc('week', r.published_at);

CREATE INDEX idx_analytics_user_week ON public.review_analytics(user_id, week DESC);

-----
-- SMS QUOTA RESET FUNCTION
-----
CREATE OR REPLACE FUNCTION public.reset_monthly_sms_quota()
RETURNS VOID AS $$
BEGIN
    UPDATE public.profiles
    SET sms_count_this_month = 0,
        sms_reset_at = date_trunc('month', NOW())
    WHERE sms_reset_at < date_trunc('month', NOW());
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

```

---

## 6. API Endpoints (Supabase Edge Functions)

### Function: poll-reviews

**Trigger:** pg\_cron every 15 minutes **Purpose:** Sync reviews from all active platforms for all users

```

// POST /functions/v1/poll-reviews
// Internal only – called by pg_cron, not exposed to clients

```

Input: none (or specific user\_id for manual trigger)

Process:

1. Fetch all active business\_profiles where platform = 'google'
2. For each profile:
  - a. Refresh token if expiry < now + 10min
  - b. Call GBP API: GET /accounts/{accountId}/locations/{locationId}/reviews
  - c. Paginate until all reviews fetched (nextPageToken)
  - d. Upsert all reviews to reviews table
  - e. Update business\_profiles.last\_polled\_at, current\_rating, review\_count
3. For newly inserted reviews: trigger push notifications

Error handling:

- 401 Unauthorized: increment poll\_error\_count, skip (will prompt reconnect after 3 errors)
- 429 Rate Limited: back off, retry next cycle
- Network failure: log error, continue to next profile

### Function: send-sms

**Trigger:** Client RPC call from app **Auth:** Requires valid JWT (user must be logged in)

```

// POST /functions/v1/send-sms
// Auth: JWT required

```

Input:

```

{
  customer_name: string,      // required
  customer_phone: string,     // required, E.164 format (+1XXXXXXXXXX)
  customer_id?: string,       // optional UUID if saving to list
  platform: 'google' | 'yelp', // default: google
  template_id?: string,       // UUID – uses default if not provided
  custom_message?: string     // override template content
}

```

Validation:

- user must have active business profile with place\_id
- Free tier: sms\_count\_this\_month < 5
- Pro tier: no limit
- customer not in opt-out list
- phone number valid format

Process:

1. Validate quota and opt-out
2. Build review link: [https://search.google.com/local/writereview?placeid={place\\_id}](https://search.google.com/local/writereview?placeid={place_id})
3. Resolve template content (system or custom)
4. Merge variables: {customer\_name}, {business\_name}, {review\_link}
5. Call Twilio: POST messages.json with To, From, Body
6. Insert review\_requests record with twilio\_message\_sid
7. Increment profiles.sms\_count\_this\_month
8. If customer\_id provided: update customers.last\_request\_at, request\_count

Output:

```

{
  success: boolean,
  request_id: string, // UUID of review_request record
  status: 'sent' | 'failed',
  error?: string
}

```

**Function: generate-reply****Trigger:** Client RPC call from app **Auth:** Requires valid JWT + Pro entitlement

```
// POST /functions/v1/generate-reply
// Auth: JWT + pro check
```

```
Input:
{
  review_id: string    // UUID of review
}
```

```
Process:
1. Verify user owns this review
2. Verify user is Pro
3. If reviews.ai_draft exists and < 24 hours old: return cached draft
4. Build Claude Haiku prompt with review content + business context
5. Call Anthropic API: claude-haiku-4-5-20251001
6. Save draft to reviews.ai_draft
7. Return draft
```

```
Output:
{
  draft: string,      // AI-generated reply text
  cached: boolean
}
```

**Function: post-reply****Trigger:** Client RPC call from app **Auth:** Requires valid JWT + Pro entitlement

```
// POST /functions/v1/post-reply
// Auth: JWT + pro check
```

```
Input:
{
  review_id: string,    // UUID of review
  reply_content: string // Text of reply (user-edited)
}
```

```
Process:
1. Verify user owns this review
2. Fetch access token for user's Google account
3. Refresh if expired
4. Call GBP API: PUT /accounts/{accountId}/locations/{locationId}/reviews/{reviewId}/reply
   Body: { comment: reply_content }
5. Update reviews: reply_content, replied_at, is_replied = true
```

```
Output:
{ success: boolean, error?: string }
```

**Function: twilio-webhook****Trigger:** HTTP POST from Twilio (no auth — webhook secret validation) **Purpose:** Handle delivery receipts and STOP replies

```
// POST /functions/v1/twilio-webhook
// No JWT — validated via Twilio signature header
```

```
Input: Twilio webhook form data
- MessageSid, MessageStatus (delivered/failed/etc)
- OR incoming message: From, Body (for STOP handling)
```

```
Process for status update:
1. Find review_request by twilio_message_sid
2. Update status accordingly
3. If delivered: set delivered_at timestamp
```

```
Process for STOP message:
1. Find customer by phone number
2. Set customers.opt_out = true, opt_out_at = now
3. All future requests to this number are blocked
```

**Function: revenuecat-webhook****Trigger:** HTTP POST from RevenueCat **Purpose:** Sync subscription status to profiles table

```
// POST /functions/v1/revenuecat-webhook
// Validated via RevenueCat authorization header
```

```
Events handled:
- INITIAL_PURCHASE: set profiles.is_pro = true
- RENEWAL: set profiles.is_pro = true
- CANCELLATION: set profiles.is_pro = false (at period end)
- EXPIRATION: set profiles.is_pro = false
- BILLING_ISSUE: send push notification about payment issue
```

```
Update profiles:
- is_pro
- revenuecat_id (from event.app_user_id)
```

**Function: send-push-notification**

**Trigger:** Supabase Database Webhook on reviews INSERT **Purpose:** Push notification for new reviews

// Internal function called by DB trigger

```
Input:
{
  review_id: string,
  user_id: string,
  type: 'new_review' | 'negative_review'
}

Process:
1. Fetch user's push tokens (is_active = true)
2. Check user's notification preferences
3. Build notification message
4. Call Expo Push API: https://exp.host/--/api/v2/push/send
5. Log notification to notifications table
```

---

## 7. State Management (Zustand Stores)

### store/auth.ts (pre-built in template)

```
interface AuthState {
  user: User | null;
  profile: Profile | null;
  session: Session | null;
  isLoading: boolean;

  signIn: (email: string, password: string) => Promise<void>;
  signUp: (email: string, password: string, businessName: string) => Promise<void>;
  signOut: () => Promise<void>;
  signInWithApple: () => Promise<void>;
  updateProfile: (updates: Partial<Profile>) => Promise<void>;
  fetchProfile: () => Promise<void>;
}
```

### store/reviews.ts (new)

```
interface ReviewsState {
  reviews: Review[];
  isLoading: boolean;
  isRefreshing: boolean;
  filter: 'all' | 'google' | 'yelp' | 'unreplied';
  hasMore: boolean;
  lastSynced: Date | null;

  fetchReviews: (reset?: boolean) => Promise<void>;
  refreshReviews: () => Promise<void>;
  loadMore: () => Promise<void>;
  updateReview: (id: string, updates: Partial<Review>) => void;
  setFilter: (filter: ReviewsState['filter']) => void;
  markReplied: (id: string, replyContent: string) => void;
}
```

### store/requests.ts (new)

```
interface RequestsState {
  requests: ReviewRequest[];
  isLoading: boolean;
  isSending: boolean;
  smsQuota: { used: number; limit: number; resetsAt: Date };

  fetchRequests: () => Promise<void>;
  sendRequest: (params: SendRequestParams) => Promise<SendResult>;
  refreshQuota: () => Promise<void>;
}
```

### store/analytics.ts (new)

```
interface AnalyticsState {
  data: AnalyticsData | null;
  isLoading: boolean;
  timeRange: '30d' | '90d' | '1y';

  fetchAnalytics: () => Promise<void>;
  setTimeRange: (range: AnalyticsState['timeRange']) => void;
}
```

### store/subscription.ts (pre-built in template — extend)

```
interface SubscriptionState {
  isPro: boolean;
  isLoading: boolean;
  expiresAt: Date | null;
  offerings: PurchasesOfferings | null;

  checkStatus: () => Promise<void>;
  purchase: (packageType: 'monthly' | 'annual') => Promise<void>;
  restore: () => Promise<void>;
}
```

```
3/11/26, 9:13 PM
  showPaywall: () => void;
}
```

store/business.ts (new)

```
interface BusinessState {
  businessProfiles: BusinessProfile[];
  isLoading: boolean;
  primaryProfile: BusinessProfile | null;

  fetchProfiles: () => Promise<void>;
  connectGoogle: () => Promise<void>;
  connectYelp: () => Promise<void>;
  disconnectPlatform: (platform: string) => Promise<void>;
}
```

8. Navigation Architecture (Expo Router)

```
app/
├── _layout.tsx                # Root layout – init services, auth guard
├── +not-found.tsx            # 404 screen
├── (auth)/                   # Unauthenticated users
│   ├── _layout.tsx          # Auth stack layout
│   ├── welcome.tsx          # Welcome + value prop
│   ├── sign-in.tsx          # Email/password sign in
│   ├── sign-up.tsx          # New account creation
│   └── forgot-password.tsx  # Password reset
├── (onboarding)/             # First-time setup (auth'd users, onboarding_complete=false)
│   ├── _layout.tsx          # Onboarding stack layout
│   ├── business-type.tsx    # Select business category
│   ├── connect-google.tsx   # GBP OAuth flow
│   └── first-request.tsx    # Send first SMS request
├── (tabs)/                   # Main app (auth'd + onboarding complete)
│   ├── _layout.tsx          # Tab bar layout
│   ├── index.tsx            # Dashboard / Review feed
│   ├── requests.tsx         # Request list + send
│   ├── analytics.tsx        # Analytics charts (Pro)
│   └── settings.tsx         # Settings + account
├── review/
│   └── [id].tsx              # Review detail + reply
├── request/
│   └── new.tsx               # New request modal
├── customers/
│   ├── index.tsx            # Customer list
│   └── [id].tsx              # Customer detail
├── templates/
│   ├── index.tsx            # Template list
│   └── [id].tsx              # Template editor
├── connect/
│   ├── google.tsx           # Google OAuth handler
│   └── yelp.tsx              # Yelp connection handler
└── paywall.tsx               # Pro upgrade screen
```

Auth Guard Logic (app/\_layout.tsx)

```
// On mount:
// 1. Check Supabase session
// 2. If no session → redirect to (auth)/welcome
// 3. If session + onboarding_complete = false → redirect to (onboarding)/business-type
// 4. If session + onboarding_complete = true → redirect to (tabs)/
```

Deep Linking

| URL                          | Screen         |
|------------------------------|----------------|
| reviewpulse://review/{id}    | review/[id]    |
| reviewpulse://requests/new   | request/new    |
| reviewpulse://settings       | tabs/settings  |
| reviewpulse://paywall        | paywall        |
| reviewpulse://connect/google | connect/google |

9. Authentication Flow

Email + Password Sign Up

- 1. User enters business name, email, password
- 2. supabase.auth.signUp() creates auth.users record
- 3. Database trigger auto-creates profiles row

4. User receives confirmation email (Supabase default)
5. On confirm: session active, redirect to onboarding

### Apple Sign In (required by App Store)

1. expo-apple-authentication presents native iOS sign-in sheet
2. Get identity token from Apple
3. Pass to supabase.auth.signInWithIdToken({ provider: 'apple', token })
4. Supabase creates/retrieves user
5. Redirect to onboarding (first time) or tabs (returning)

### Session Persistence

- AsyncStorage via @supabase/storage-file-system-adapter
- Auto-refresh: Supabase SDK handles token refresh
- Session check on app foreground (AppState listener)

### Google OAuth (GBP connection — separate from auth)

- expo-auth-session with useAuthRequest
- Redirect URI: reviewpulse://connect/google
- Scope: https://www.googleapis.com/auth/business.manage
- Token exchange in Edge Function (never in app code)
- Access + refresh token stored encrypted in Supabase

---

## 10. Payment Integration (RevenueCat)

### Products

| Product ID     | Price         | Trial  | Platform      |
|----------------|---------------|--------|---------------|
| rp_pro_monthly | \$14.99/month | 7 days | iOS + Android |
| rp_pro_annual  | \$119.99/year | 7 days | iOS + Android |

### RevenueCat Configuration

App: ReviewPulse  
Bundle ID: com.reviewpulse.app  
Entitlement: pro  
- Products: rp\_pro\_monthly, rp\_pro\_annual

Packages:  
- Default: rp\_pro\_monthly (shown in paywall)  
- Annual: rp\_pro\_annual

Offerings:  
- default: both packages shown

### Client Implementation

```
// Initialize on app start
await Purchases.configure({ apiKey: REVENUECAT_PUBLIC_KEY });

// Purchase
const offerings = await Purchases.getOfferings();
const { customerInfo } = await Purchases.purchasePackage(package);
const isPro = customerInfo.entitlements.active['pro'] !== undefined;

// Restore
const { customerInfo } = await Purchases.restorePurchases();
```

### Server-Side Validation

RevenueCat webhook → revenuecat-webhook Edge Function → updates profiles.is\_pro Double validation: client checks RevenueCat SDK, server checks DB flag for Pro features

### Paywall Trigger Points

| Feature Attempted            | Paywall Shown          |
|------------------------------|------------------------|
| 6th SMS request in a month   | ✓ (with usage counter) |
| “Draft Reply” button tap     | ✓ (on first attempt)   |
| Yelp connect tap in settings | ✓                      |
| Analytics tab (when Free)    | ✓ (blurred overlay)    |
| Bulk CSV import              | ✓                      |

---

## 11. Testing Strategy

### Unit Tests

```
tests/
├── utils/
│   ├── phone-formatter.test.ts    # Phone number formatting/validation
│   ├── sms-template.test.ts      # Merge tag resolution
│   ├── review-link.test.ts       # Google review link generation
│   └── quota-check.test.ts        # SMS quota logic
├── stores/
│   ├── reviews.test.ts           # Review store CRUD
│   ├── requests.test.ts          # Request store send flow
│   └── subscription.test.ts       # Pro gate logic
└── edge-functions/
    ├── send-sms.test.ts           # SMS send + validation
    ├── generate-reply.test.ts     # AI draft prompt construction
    └── twilio-webhook.test.ts     # Opt-out handling
```

Component Tests

```
tests/components/
├── ReviewCard.test.tsx           # Rating display, reply status
├── RequestForm.test.tsx          # Phone validation, template merge
├── AnalyticsChart.test.tsx       # Data rendering, pro gate
└── PaywallScreen.test.tsx        # Purchase flow, restore
```

Integration Tests

```
tests/integration/
├── auth-flow.test.ts             # Sign up → onboarding → tabs
├── review-request-flow.test.ts   # Send SMS → webhook → status update
└── google-oauth-flow.test.ts     # Connect → reviews sync
```

Test Commands

```
# Unit tests
npx jest tests/utils/ tests/stores/

# Component tests
npx jest tests/components/

# All tests
npx jest --coverage
```

12. Build & Deploy

Environment Variables

```
# Supabase
EXPO_PUBLIC_SUPABASE_URL=https://[project-id].supabase.co
EXPO_PUBLIC_SUPABASE_ANON_KEY=[anon-key]
SUPABASE_SERVICE_KEY=[service-key]           # Edge Functions only

# RevenueCat
EXPO_PUBLIC_REVENUECAT_IOS_KEY=[ios-public-key]
EXPO_PUBLIC_REVENUECAT_ANDROID_KEY=[android-public-key]

# Sentry
EXPO_PUBLIC_SENTRY_DSN=[dsn]

# PostHog
EXPO_PUBLIC_POSTHOG_API_KEY=[key]

# Edge Function Secrets (set in Supabase Dashboard → Settings → Secrets)
GOOGLE_CLIENT_ID=[client-id]
GOOGLE_CLIENT_SECRET=[client-secret]
TWILIO_ACCOUNT_SID=[sid]
TWILIO_AUTH_TOKEN=[token]
TWILIO_PHONE_NUMBER=[+1XXXXXXXXXX]
ANTHROPIC_API_KEY=[sk-ant-...]
REVENUECAT_SECRET_KEY=[secret-key]
TOKEN_ENCRYPTION_KEY=[32-byte-hex]           # For Google token encryption
```

EAS Build Profiles

```
{
  "build": {
    "development": {
      "developmentClient": true,
      "distribution": "internal",
      "ios": { "simulator": true }
    },
    "preview": {
      "distribution": "internal",
      "android": { "buildType": "apk" }
    },
    "production": {
      "autoIncrement": true,
      "env": {
        "NODE_ENV": "production"
      }
    }
  },
  "submit": {
```

```

    "production": {
      "ios": {
        "appleId": "[apple-id]",
        "ascAppId": "[app-store-connect-app-id]",
        "appleTeamId": "[team-id]"
      },
      "android": {
        "serviceAccountKeyPath": "./credentials/google-play-key.json",
        "track": "internal"
      }
    }
  }
}

```

## Build Commands

```

# Development build (iOS simulator)
eas build --profile development --platform ios

```

```

# Preview build (for TestFlight / internal)
eas build --profile preview --platform all

```

```

# Production build
eas build --profile production --platform all

```

```

# Submit to stores
eas submit --profile production --platform all

```

## OTA Updates (Expo Updates)

```

// app/_layout.tsx – check for updates on app foreground
import * as Updates from 'expo-updates';

useEffect(() => {
  const subscription = AppState.addListener('change', async (state) => {
    if (state === 'active') {
      const update = await Updates.checkForUpdateAsync();
      if (update.isAvailable) {
        await Updates.fetchUpdateAsync();
        await Updates.reloadAsync();
      }
    }
  });
  return () => subscription?.remove();
}, []);

```

# 13. App Store Submission

## App Store Metadata

**Name:** ReviewPulse: Google Reviews **Subtitle:** Request, Monitor & Respond **Category:** Business (Primary), Productivity (Secondary) **Age Rating:** 4+ (no objectionable content)

### Description (4000 char limit):

Stop losing customers to businesses with more reviews.

ReviewPulse is the mobile-native review manager built for solo business owners who can't afford \$400/month agency tools but need Google review

### SEND SMS REVIEW REQUESTS IN SECONDS

After every job, type your customer's name and phone number – ReviewPulse sends them a direct link to your Google Business Profile review pag

### MONITOR EVERY REVIEW IN REAL TIME

See new reviews the moment they're posted. Get instant push notifications for new feedback – especially critical when a 1-star review appears

### REPLY WITH AI IN ONE TAP

Struggling to find the right words? Tap "Draft Reply" and ReviewPulse generates a professional, personalized response in 4 seconds using AI. ⚡

### BUILT FOR YOUR BUSINESS TYPE

Templates designed for trades (plumbers, electricians, HVAC), beauty (barbers, stylists, estheticians), wellness (massage therapists, chiropr

### NO CONTRACTS. NO SURPRISES.

Unlike Podium (\$399/month) or Birdeye (\$299/month), ReviewPulse Pro is \$14.99/month with no annual contract. Cancel anytime from Settings.

### WHAT YOU GET FREE:

- Connect 1 Google Business Profile
- View all your reviews
- 5 SMS review requests per month
- Basic rating analytics

### REVIEWPULSE PRO (\$14.99/MONTH):

- Unlimited SMS review requests
- AI-drafted reply suggestions
- Yelp + Trustpilot monitoring
- Full analytics: rating trends, velocity, response rate
- Push notifications for every new review
- Custom SMS templates for your industry
- Bulk import customer contacts (CSV)
- Suspicious review flagging

### 7-DAY FREE TRIAL. CANCEL ANYTIME.

No annual contract. No hidden fees. If it doesn't help you get more reviews, cancel in one tap.



**Keywords** (100 chars max): google review, review management, sms review, get reviews, reputation, small business, review request

## Privacy Nutrition Labels

**Data Used to Track You:** None **Data Linked to You:** - Contact Info (email, name, phone) - Location (business address) - Financial Info (purchase history) - Usage Data

**Data Not Linked to You:** - Customer contact info (collected by user for their own business use)

## Screenshots Plan (iPhone 6.9", required)

1. **Dashboard** — Shows review feed with star ratings, reply status badges, and new review notification
2. **Send Request** — New request form with customer name, phone, and SMS preview showing personalized message
3. **AI Reply** — Review detail with AI-drafted response being edited before posting
4. **Analytics** — Charts showing rating trend upward with key metrics: response rate 89%, 14 new reviews
5. **Paywall** — Pro features list with “No contract” emphasis and pricing
6. **Onboarding** — Business type selection grid with clean, welcoming UI

# 14. Implementation Checklist

## Sprint 1: Foundation

- ☐ Set up Expo project from base template: `setup-new-app.sh reviewpulse "ReviewPulse"`
- ☐ Configure `app.config.ts` with ReviewPulse bundle ID (`com.reviewpulse.app`)
- ☐ Create Supabase project, run base + reviewpulse migrations
- ☐ Configure all environment variables in `.env`
- ☐ Install dependencies: `react-native-purchases, react-native-gifted-charts, papaparse, @types/papaparse`
- ☐ Verify template base builds: `npx expo export --platform web`

## Sprint 2: Design System

- ☐ Invoke frontend-design skill → generate full design system spec
- ☐ Write `DESIGN.md` with color tokens, typography, spacing, component specs
- ☐ Configure `tailwind.config.js` with ReviewPulse design tokens
- ☐ Create `theme.ts` with all design tokens as constants
- ☐ Build shared components: Button, Card, Badge, Input, SkeletonLoader
- ☐ Build StarRating component (display + interactive variants)

## Sprint 3: Authentication

- ☐ Implement sign-in screen with email/password
- ☐ Implement sign-up screen with business name field
- ☐ Implement forgot password screen
- ☐ Add Apple Sign In (required for App Store)
- ☐ Wire auth store to screens
- ☐ Implement welcome screen with value prop + CTAs
- ☐ Test full auth flow (sign up → email confirm → sign in)

## Sprint 4: Onboarding

- ☐ Business type selection screen (8 category grid)
- ☐ Google Business Profile OAuth screen (connect flow)
- ☐ `connect-google` Edge Function (token exchange + location fetch)
- ☐ Location picker (for users with multiple locations)
- ☐ First request prompt screen
- ☐ Update `profiles.onboarding_complete` on finish
- ☐ Auth guard redirects to onboarding if `onboarding_complete = false`

## Sprint 5: Review Monitoring Core

- ☐ `poll-reviews` Edge Function (GBP API + upsert)
- ☐ Google OAuth token refresh logic in Edge Function
- ☐ Review feed screen (`tabs/index.tsx`) with FlatList
- ☐ ReviewCard component (author, rating, excerpt, reply status badge)
- ☐ Supabase Realtime subscription for live updates
- ☐ Pull-to-refresh on review feed
- ☐ Empty state (no reviews yet) + loading skeleton

## Sprint 6: Review Detail + Reply

- ☐ Review detail screen (`review/[id].tsx`)
- ☐ `generate-reply` Edge Function (Claude Haiku + prompt)
- ☐ AI draft loading state (shimmer in text area)
- ☐ `post-reply` Edge Function (GBP API reply PUT)

- ☐ Reply posted success animation
- ☐ Review flagging UI + flag reasons picker
- ☐ Pro gate on “Draft Reply” button → paywall

### Sprint 7: SMS Review Requests

- ☐ send-sms Edge Function (Twilio API + quota validation)
- ☐ twilio-webhook Edge Function (delivery receipts + STOP)
- ☐ Request list screen (tabs/requests.tsx)
- ☐ New request modal (request/new.tsx)
- ☐ Customer name + phone input with formatting
- ☐ Template selector (system templates first pass)
- ☐ SMS message preview (live-updating)
- ☐ Free tier quota display (X of 5 used)
- ☐ Send success animation + haptic

### Sprint 8: Subscription Integration

- ☐ RevenueCat SDK initialization in root layout
- ☐ Paywall screen (paywall.tsx) with monthly/annual toggle
- ☐ Monthly + annual purchase flow
- ☐ 7-day free trial handling
- ☐ Restore purchases
- ☐ revenuecat-webhook Edge Function → updates profiles.is\_pro
- ☐ useIsPro() hook for feature gating
- ☐ Pro gates: AI reply, 6th SMS, Yelp, analytics, CSV import
- ☐ Post-purchase success animation

### Sprint 9: Analytics Dashboard

- ☐ Analytics materialized view + refresh function
- ☐ Analytics tab screen (tabs/analytics.tsx)
- ☐ Rating trend line chart (30d/90d/1y)
- ☐ Review velocity bar chart
- ☐ Response rate donut chart
- ☐ Platform breakdown pie chart
- ☐ AI-generated insights (Claude Haiku 1-sentence summary)
- ☐ Pro gate with blurred overlay + upgrade CTA

### Sprint 10: Push Notifications

- ☐ send-push-notification Edge Function
- ☐ Database webhook trigger on reviews INSERT
- ☐ Push notification permission request in onboarding
- ☐ Notification deep link handler (review/[id])
- ☐ In-app notification center (bell icon → list)
- ☐ Notification preference toggles in settings

### Sprint 11: Customers + Templates

- ☐ Customer list screen (customers/index.tsx)
- ☐ Customer detail screen (customers/[id].tsx)
- ☐ Save customer after sending first request (prompt)
- ☐ CSV import flow (expo-document-picker + papaparse)
- ☐ Template list screen (templates/index.tsx)
- ☐ Template editor screen (templates/[id].tsx)
- ☐ Merge tag buttons in editor ({customer\_name}, etc.)
- ☐ System templates seeded in DB migration

### Sprint 12: Settings + Polish

- ☐ Settings screen (tabs/settings.tsx) — all sections
- ☐ Connected platforms section (Google/Yelp status + reconnect)
- ☐ Notification toggles wired to user\_settings table
- ☐ Dark mode — full review pass on all screens
- ☐ Haptic feedback on all interactive elements
- ☐ Loading skeletons for all async lists
- ☐ Empty states for all lists (with illustrations)
- ☐ Error boundaries on all screens
- ☐ Offline banner component

### Sprint 13: Yelp Integration (Pro)

- ☐ Yelp Fusion API setup in Edge Function environment
- ☐ Yelp business search by name+address
- ☐ Add Yelp to poll - reviews Edge Function
- ☐ Yelp connect flow in settings
- ☐ “Reply on Yelp” button (opens Yelp app/browser) instead of in-app reply

**Sprint 14: Final QA + App Store**

- ☐ TypeScript strict mode — fix all any types
- ☐ npx expo lint — fix all warnings
- ☐ Security scan — no hardcoded credentials
- ☐ Test on real iOS device (not just simulator)
- ☐ Test on Android device
- ☐ Write privacy policy (privacy-policy.md)
- ☐ Write App Store description + keywords
- ☐ Generate app icon (1024x1024)
- ☐ Generate 6 iPhone 6.9” screenshots
- ☐ Configure App Store Connect metadata
- ☐ Production EAS build
- ☐ TestFlight internal review
- ☐ Submit for App Store review

**Roadmap Progress Tracker**

Total checklist items: 87 Completed: 0 Progress: 0%

**Phase Summary**

| Sprint                           | Items | Status      |
|----------------------------------|-------|-------------|
| Sprint 1: Foundation             | 6     | Not Started |
| Sprint 2: Design System          | 6     | Not Started |
| Sprint 3: Authentication         | 7     | Not Started |
| Sprint 4: Onboarding             | 7     | Not Started |
| Sprint 5: Review Monitoring      | 7     | Not Started |
| Sprint 6: Review Detail + Reply  | 7     | Not Started |
| Sprint 7: SMS Review Requests    | 9     | Not Started |
| Sprint 8: Subscription           | 9     | Not Started |
| Sprint 9: Analytics              | 8     | Not Started |
| Sprint 10: Push Notifications    | 6     | Not Started |
| Sprint 11: Customers + Templates | 8     | Not Started |
| Sprint 12: Settings + Polish     | 8     | Not Started |
| Sprint 13: Yelp Integration      | 5     | Not Started |
| Sprint 14: Final QA + App Store  | 15    | Not Started |

Last updated: 2026-03-11 — Phase 4 Roadmap complete. Ready for Phase 5 (Design).